



LEKTION 12

KLASSE DIAGRAM

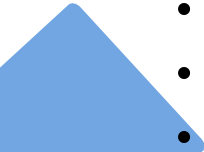
MÅL

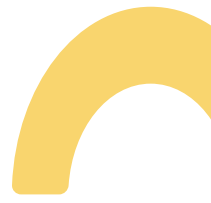
EFTER ENDT UNDERVISNING KAN DEN STUDERENDE:

- Gå fra krav til Kode
- Bygge klasse diagrammer
- Benytte de 4 søjler i OOP på klasse diagrammer.
- Kende til forskellige relationer i klassediagrammet.



AGENDA

- 
- Recap
 - Klasser
 - Relationer
 - Multiplicity
 - Opgave
 - Næste gang





UCCASE DIAGRAMMER

SEKVENSS DIAGRAMMER

OOP + KLASSER

KLASSE DIAGRAMMER

INDSÆT DIAGRAM FRA UML SPEC

Fra sekvens til klassediagram

- Sekvensdiagram: viser samspil over tid
(hvem kalder hvem)
- Klassediagram: viser strukturen bag samspillet
(klasser + relationer)
- Overgang: objekter i sekvensdiagrammet bliver til
klasser i klassediagrammet
(Klasser)
- Metodekald i sekvensdiagrammet peger på
ansvar/operationer i klasserne
(Attributter og Metoder)

KODE

MERMAID KODE FOR KLASSEDIAGRAM. EN KLASSE MED ACCESS MODIFIER

```
classDiagram
    class Animal{
        # _sound : string
        - _feather : bool
        + beak : byte
        ~ legs : byte

        + Animal(legs) : void
        - GetFeather() : bool
        + GetSound() : string
        + GetLeg() : byte
        + GetBeak() : byte
    }
```

Public: Alle har adgang

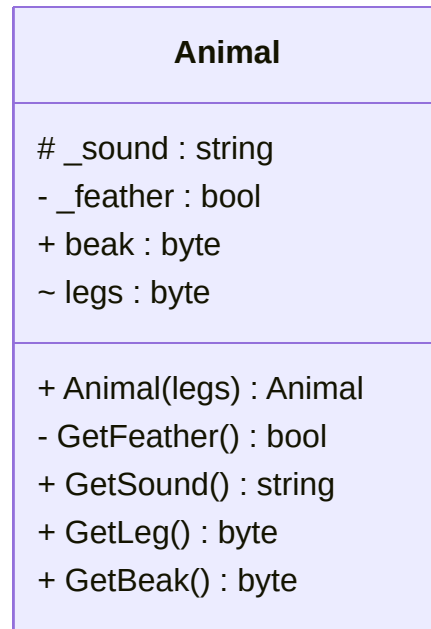
Private: Kun en selv har adgang

Protected: Kun sig selv og børn har adgang

Internal: Kun projektet har adgang

DIAGRAM

DIAGRAMMET FOR KLASSEDIAGRAM. EN KLASSE MED ACCESS MODIFIER



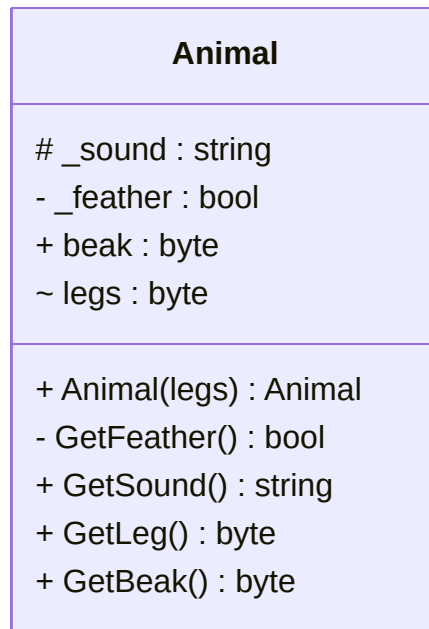
KODE

FOR FULD IMPLEMENTERING SE KODEN FRA
GITHUB

```
class Animal // Class animal {}  
{  
    protected string _sound; // # _sound : string  
    private bool _feather; // - _feather : bool  
    public byte beak; // + beak : byte  
    internal byte _legs; // ~ legs : byte  
  
    public Animal(byte legs) // + Animal(legs) : void  
    {  
        _sound = "Quack";  
        _feather = false;  
        beak = 1;  
        _legs = legs;  
    }  
  
    private bool GetFeather() { } // - GetFeather() : bool  
    public string GetSound() { } // + GetSound() : string  
    public byte GetLeg() { } // + GetLeg() : byte  
    public byte GetBeak() { } // + GetBeak() : byte  
}
```

DIAGRAM

FOR MERMAID KODE SE TIDLIGERE SLIDE
OG/ELLER GITHUB



RELATIONER

- Associationer
Has-a relation. Bil HAR-EN motor. Hest HAR-EN hale.
- Aggregation
Viser en speciel association mellem to klasser - IKKE nedarvning. Den ene klasse ejer ikke den anden.
- Composit
Lifeline dependent. Viser en speciel association mellem to klasser - IKKE nedavning. Den ene klasse ejer den anden.
- Inheritance/generalization
Nedarvning, is-a relation. Bil ER-ET transportmiddel. Hest ER-ET dyr.
- Realization
 - Abstract
is-a relation. En abstrakt klasse kan indeholde metoder med implementation og abstract. En abstract metode skal eksplicit deklareres. Child klassen skal implementere alle abstrakte metoder.
 - Interface
can-do relation. En interface definere en kontrakt. En child af et interface skal implementere alle abstrakte metoder. Interface metoder er implicit abstrakte medmindre der eksistere en default implementation.

AGGREGATION

Se program kode i github under code

Se mermaid kode på GitHub ller rå markdown fil.

```
class Course
{
    Student[] students = new Student[25];
    public Course(Student[] students)
    {
        this.students = students;
    }
}

class Student
{
    private string _name = "";
    private byte _age = 0;

    public Student(string name, byte age)
    {
        _name = name;
        _age = age;
    }
}
```

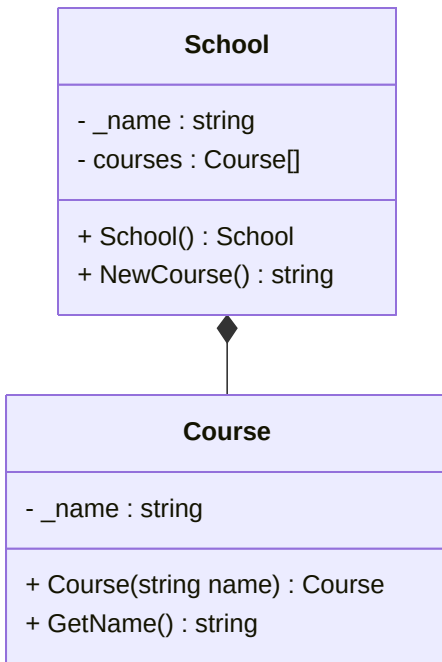
COMPOSITION

Se program kode i github under code

```
class School
{
    public string NewCourse(string courseName)
    {
        for (int i = 0; i < _courses.Length; i++)
        {
            if (_courses[i] == null)
            {
                _courses[i] = new Course("Course " + courseName);
                return _courses[i].GetName();
            }
        }
        return null; // No more courses can be added
    }
}

class Course
{
    private string _name;
    public Course(string name){}
```

Se mermaid kode på GitHub.



INHERITANCE/GENERALIZATION

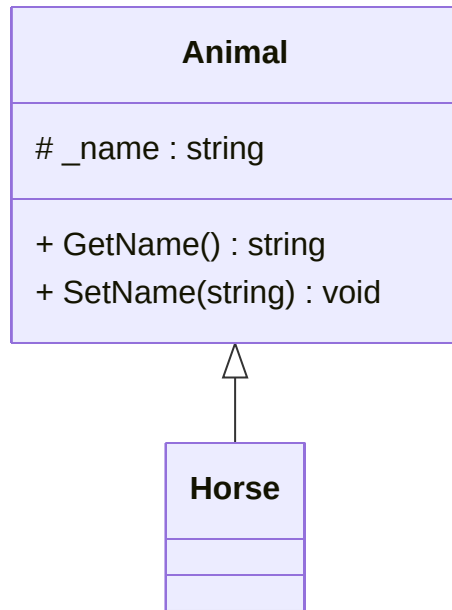
Se program kode i github under code

```
class Animal
{
    protected string _name = "";

    public string GetName()
    {
        return _name;
    }
    public void SetName(string name)
    {
        _name = name;
    }
}

class Horse : Animal
{
}
```

Se mermaid kode på GitHub ller rå markdown fil.



REALIZATION - ABSTRACT

is-a relation. En abstrakt klasse kan indeholde metoder med implementation og abstrakt

Se program kode i github under code

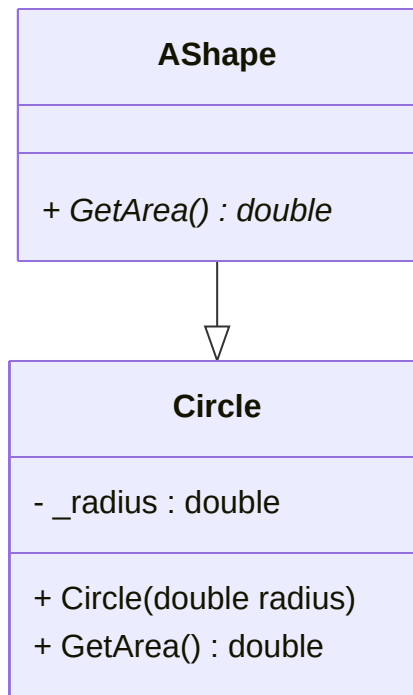
Se mermaid kode på GitHub

```
abstract class AShape
{
    public abstract double GetArea(); // Child handle implementation
}

class Circle : AShape
{
    private double _radius;

    public Circle(double radius)
    {
        _radius = radius;
    }

    public override double GetArea() // Override the parent method
    {
        return Math.PI * _radius * _radius;
    }
}
```



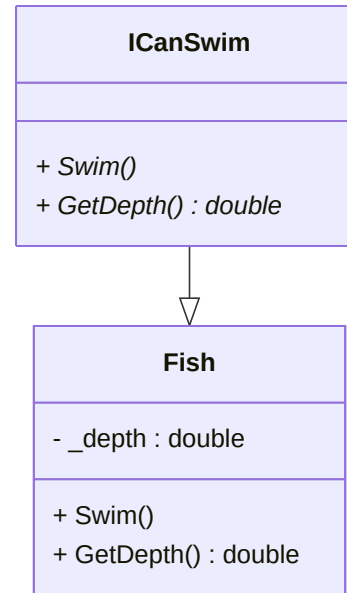
REALIZATION - INTERFACE

can-do relation. En interface definere en kontrakt.

Se program kode i github under code

Se mermaid kode på GitHub

```
interface ICanSwim
{
    void Swim();
    double GetDepth();
}
class Fish : ICanSwim
{
    private double _depth;
    public void Swim()
    {
        Console.WriteLine("Fish is swimming!");
        Console.WriteLine("Splash splash");
    }
    public double GetDepth()
    {
        return _depth;
    }
}
```



MULTIPLICITY

KARDINALITET I DATAFORSTÅELSE.

Vi har allerede snakket kort om det i UseCase diagram.

Beskriver hvor mange objekter der er/kan være i hver relation. Eksempelvis, har en bil en motor. En el-bil kan have 4 motor.

Notation:

- 0
- 1
- *

Alle tænkelig kombinationer heri mellem

MULTIPLICITY

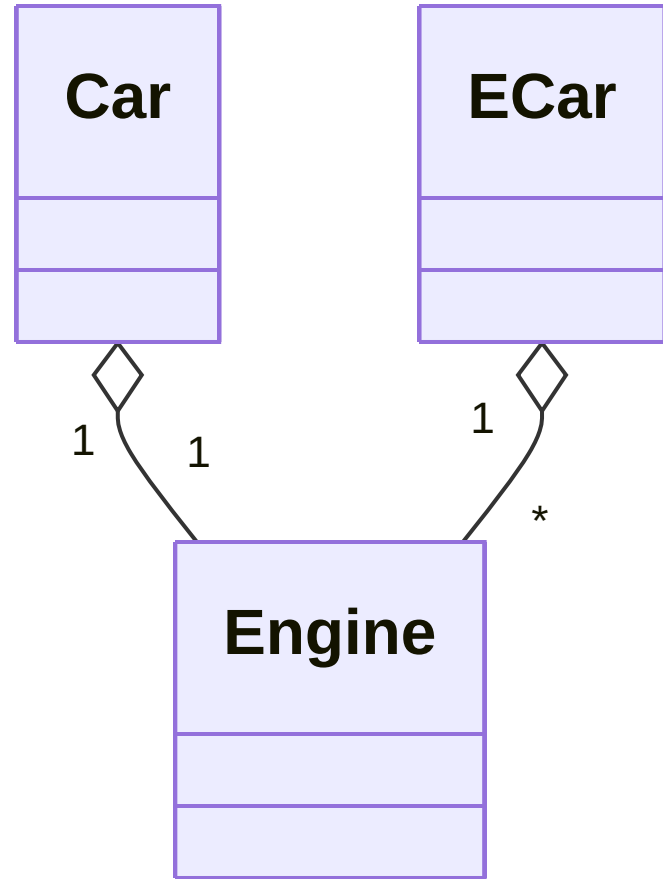
```
class Engine
{
    public string Model { get; set; }
    public Engine(string model) { Model = model; }
}

class Car
{
    private Engine _engine;

    public Car(Engine engine)
    {
        _engine = engine;
    }
}

class ECar
{
    private Engine[] _engines;

    public ECar(Engine[] engines)
    {
        _engines = engines;
    }
}
```



<https://forms.cloud.microsoft/e/dSmSFWfHhk>



OPGAVE 1 - FRA SEKVENSTIL KLASSE

I får følgende beskrivelse fra et sekvensdiagram:

- User kalder createOrder()
- OrderService kalder validateOrder()
- OrderService kalder saveOrder() i Database
- Database returnerer OrderId

OPGAVE

1. Identificér hvilke klasser der skal eksistere
2. Identificér hvilke metoder der skal være i hver klasse
3. Tegn et simpelt klassediagram i Mermaid
4. Implementér skeleton-klasser i C#

OPGAVE 2 - RELATIONER

Udvid systemet:

- En Order har én Customer
- En Customer kan have mange Order
- En Order indeholder flere OrderLine
- En OrderLine indeholder ét Product

OPGAVE

Identificér relationstyper:

1. Association?
2. Aggregation?
3. Composition?

Tilføj multiplicity korrekt

1. Tegn opdateret klassediagram i Mermaid
2. Implementér relationerne i C#

OPGAVE 3 - GENERALIZATION & REALIZATION

Udvid domænet:

Product kan være:

- PhysicalProduct
- DigitalProduct

Begge skal kunne:

- CalculatePrice()

Ekstra krav:

Digitale produkter har ingen lagerstatus Fysiske produkter har lagerantal

OPGAVE

1. Brug nedarvning korrekt
2. Brug abstrakt klasse eller interface (argumentér for valg)
3. Tegn klassesdiagram med generalization/realization
4. Implementér i C#

OPGAVE 4 - SAMLET MODEL + DESIGNOVERVEJELSE

Tilføj betalingssystem:

1. En Order skal kunne betales

Der findes:

- CreditCardPayment
- MobilePayment
- InvoicePayment

Krav:

1. Alle betalingstyper skal implementere `ProcessPayment()`
2. Order må ikke kende til konkrete betalingstyper
3. Systemet skal kunne udvides med nye betalingstyper uden at ændre Order

OPGAVE 4 - SAMLET MODEL + DESIGNOVERVEJELSE

OPGAVE

1. Design relationen korrekt (interface eller abstract?)
2. Tegn komplet klassediagram med:
3. Relationer
4. Multiplicity
5. Generalization
6. Realization
7. Implementér designet i C#

Argumentér kort:

- Hvilke OOP-søjler bruges?
- Hvis man laver det forkert, hvilke OOP søjler bryder man så ?